Nighat Medical Complex AI API & Model Audit

> **Active response engine: RULE-BASED**
> **No external language model is called.**

1. Executive Summary

The inspected patient chatbot is a deterministic application: keyword matching, predefined multilingual messages, verified MongoDB clinic information and a booking state machine. No active OpenAI, Gemini, Groq, Anthropic or other external LLM integration was found. The interface label "AI Assistant" is not evidence of an LLM. [backend/services/conversationEngine.js:219] [backend/services/conversationEngine.js:390] [backend/services/knowledgeService.js:45]

The local configuration contains AI_PROVIDER=built-in and AI_MODEL=gemini-1.5-flash. These settings are not referenced by the active response code. gemini-1.5-flash is an unused configured identifier, not the model generating responses. AI_API_KEY is empty. No source code, environment setting, database connection or booking behavior was changed for this audit.

Investigation Item	Finding
AI Provider	No active external provider found; built-in application logic
AI Model	None active. Unused configured identifier: gemini-1.5-flash
AI API Endpoint	Not Found. Meta Graph API is transport, not an LLM endpoint.
API Key Variable	AI_API_KEY exists but is empty and unused
AI Service File	backend/services/conversationEngine.js; backend/services/leadAiService.js
Response Type	Rule-Based; not an active LLM/rules hybrid
Website Integration	Verified in source and offline engine greeting check
WhatsApp Integration	Shared engine verified; live Cloud API delivery Not Verified
Knowledge Base	MongoDB KnowledgeBase, key nighat_medical_complex

Scope: active application source, backend/frontend manifests and lockfiles, routes, controllers, services, configuration references and local environment presence checks. Historical backups and installed dependency internals are not evidence of active application integration.

2. AI Provider Identification

OpenAI: no SDK import, endpoint request or active adapter found. Google Gemini: mentioned in an optional-adapter comment and unused model configuration, but no implementation found. Groq, Anthropic Claude and other external LLM providers: no SDK imports, model invocations or inference endpoints found. The backend dependencies contain Express, Mongoose and supporting utilities, but no provider AI SDK. [backend/package.json:19]

Absence of an SDK alone would not prove absence of an API: the audit also traced fetch calls, imports, exported functions and the message-generating branches. The only outbound HTTP call found in active backend services is the WhatsApp Meta Graph message transport. [backend/services/metaWebhookService.js:219]

3. Exact AI Model

Active inference model	Not Found; no model inference call
AI_PROVIDER local value	built-in
AI_MODEL local value	gemini-1.5-flash
Meaning of the configured model	Unused configuration; no active code reads AI_MODEL
Optional LLM adapter	callOptionalLLM() returns null; no call sites found

Evidence: the same nonsecret example settings appear in backend/.env.example:17-19. The local backend/.env provider/model identifiers were inspected through an allowlist; no key or password values were printed. Configuration can be stale even when an identifier looks like a valid model name. [backend/services/conversationEngine.js:219]

4. AI API Configuration

Variable	Local file status / active code usage
AI_PROVIDER	Present: built-in / not referenced
AI_MODEL	Present: gemini-1.5-flash / not referenced
AI_API_KEY	Present, empty / not referenced
OPENAI_API_KEY	Absent / no reference found
GEMINI_API_KEY	Absent / no reference found
GOOGLE_API_KEY	Absent / no reference found
GROQ_API_KEY	Absent / no reference found
ANTHROPIC_API_KEY	Absent / no reference found

Local backend/.env is the inspected configuration source. Environment variables injected into an independently deployed server or shell were not inspected and are Not Verified. Even a configured secret would not activate the current null adapter. No provider credential validity or billing request was attempted.

5. AI Response Generation Workflow

For a fresh, non-manual session with no active booking workflow, the patient sends "Hi":

1. Frontend send	sendPatientMessageToBackend(text, payload, action, ...) sends JSON with session ID, text, language and client message ID. frontend/patient/app.js:2794
2. HTTP request	POST /api/staff/chat/message. frontend/patient/app.js:2808
3. Express routing	server.js mounts /api/staff; staffRoutes maps /chat/message to handlePatientMessage. backend/server.js:63; backend/routes/staffRoutes.js:20
4. Controller	handlePatientMessage stores/updates the conversation and calls processConversationMessage with channel website. backend/controllers/staffController.js:83; backend/controllers/staffController.js:225
5. Knowledge context	processConversationMessage wraps execution with knowledge.withKnowledge; getRawKnowledge reads the MongoDB knowledge record when connected. backend/services/conversationEngine.js:1150; backend/services/knowledgeService.js:15
6. Response generation	processConversationWithContext matches the greeting regular expression, then returns WELCOME_GREETING and language options. No AI API call occurs. backend/services/conversationEngine.js:390; backend/services/conversationEngine.js:392
7. Message processing	recordAiMessage appends the assistant message, options and state, then saves the session when connected. backend/services/conversationEngine.js:1021
8. Return and render	Controller returns JSON reply/messages/options; frontend sorts messages and calls renderMessageItem. backend/controllers/staffController.js:237; frontend/patient/app.js:2834

A "Hi" during an active workflow or human takeover can take a different branch. The greeting branch requires no active workflow; manual mode is checked earlier. These conditions prevent an incorrect claim that every "Hi" always yields the welcome message. [backend/services/conversationEngine.js:304] [backend/services/conversationEngine.js:391]

Offline verification performed during this audit: invoked the exported engine for fresh website and whatsapp sessions, without connecting MongoDB. Both responses exactly matched the predefined English greeting and produced LANGUAGE_SELECTION with two options. A blocked fetch stub recorded zero outbound calls. No production patient or appointment writes occurred. This verifies the local engine behavior, not a public deployment or WhatsApp delivery.

6. Source Code Evidence

Line numbers refer to this local source snapshot. Open these paths relative to the project root; no source files were edited during the audit.

backend/services/conversationEngine.js:219	Optional adapter is a no-op, not an inference integration.
backend/services/conversationEngine.js:140	Greeting text originates in the knowledge module constants.
backend/services/conversationEngine.js:390	Regex recognizes Hi/Hello/Salam and other greeting forms.
backend/services/conversationEngine.js:450	Keyword-selected knowledge answers precede workflow processing.
backend/services/conversationEngine.js:470	FAQ fallback uses exact normalized question and language matching.
backend/services/conversationEngine.js:479	Booking prompts are driven by workflow state.
backend/services/conversationEngine.js:921	Confirmed bookings delegate to the real booking service.
backend/services/knowledgeService.js:15	Knowledge is read from MongoDB, not vector retrieval.
backend/services/knowledgeService.js:18	Verified/active filtering controls patient-facing facts.
backend/services/knowledgeService.js:45	Regular expressions route natural text to information topics.
backend/services/leadAiService.js:14	Lead/WhatsApp text uses the same engine as website.
backend/services/metaWebhookService.js:219	Graph API sends WhatsApp messages; it does not generate text.
backend/services/metaWebhookService.js:212	Development credentials can cause simulated outbound delivery.

Search coverage included active .js files, configuration templates and package manifests/lockfiles. Searches checked provider names, SDK imports, inference endpoints, fetch/HTTP calls, chat completions, Responses APIs, AI environment variable names, model settings and optional adapter references. Provider comments and UI labels were treated as descriptions, not proof of inference.

7. Rule-Based vs AI-Generated Responses

Response category	Actual mechanism
Greeting and language menu	Prewritten English/Urdu constants plus regex and payload branches.
Doctor, hours, services, fees	Keyword topic routing and formatting verified knowledge fields.
Appointment booking	State machine: name, phone, date, slot, review, confirmation; real availability and database service.
Emergency and handover	Keyword lists and fixed warning/handover templates; not a diagnostic model.
Unknown questions	Exact normalized FAQ match; otherwise fixed unknown-information/staff response.
Manual mode	Human/staff workflow, not LLM generation.
External LLM-generated replies	None found in the active code.

Evidence: backend/services/conversationEngine.js:390; backend/services/conversationEngine.js:397; backend/services/conversationEngine.js:320; backend/services/conversationEngine.js:343; backend/services/conversationEngine.js:470; backend/services/knowledgeService.js:37. Rule-based automation can make genuine appointments. A real database write is not evidence of a language model.

8. Knowledge Base Integration

KnowledgeBase.findOne({key: KEY}).lean() loads the clinic record using the key nighat_medical_complex when Mongoose is connected. AsyncLocalStorage supplies a per-request approved knowledge snapshot. Disconnected or failed reads return conservative defaults. This audit did not connect to MongoDB or modify any knowledge record. [backend/services/knowledgeService.js:6] [backend/services/knowledgeService.js:13] [backend/services/knowledgeService.js:68]

Only verified, active doctors/services and verified FAQs are exposed. Unverified location, contacts, hours and fees are filtered. answer() formats these fields; matchTopic() decides which topic to show. No embeddings, vector database search, model prompt submission or generative retrieval pipeline was found. [backend/services/knowledgeService.js:18] [backend/services/knowledgeService.js:37] [backend/services/knowledgeService.js:45]

SYSTEM_INSTRUCTIONS is a string exported by the knowledge service and lead service. No active model call consumes it as a system prompt. The label or presence of a prompt-like string does not enforce behavior through an LLM. [backend/services/knowledgeService.js:7] [backend/services/leadAiService.js:20]

9. Website and WhatsApp Integration

Website: handlePatientMessage calls processConversationMessage with channel website. WhatsApp/lead intake: the Meta webhook routes WhatsApp text to processIncomingLeadMessage, which invokes the same exported engine with channel whatsapp. Both therefore use the same rule-based response implementation and approved knowledge, with no shared external AI provider or model. [backend/controllers/staffController.js:227] [backend/services/metaWebhookService.js:237] [backend/services/leadAiService.js:14]

Transport: sendWhatsAppCloudMessage formats the result with formatWhatsAppPayload and can POST to https://graph.facebook.com/v19.0/{phoneId}/messages. This is a WhatsApp delivery API, not a text-generation API. Local Meta token/phone/app-secret/verification settings are development placeholders. The sender explicitly returns in its development-token branch; production delivery is Not Verified. [backend/services/metaWebhookService.js:208] [backend/services/metaWebhookService.js:212] [backend/services/metaWebhookService.js:219]

Important implementation gap: processWhatsAppPayload stores the interactive button/list ID inside metadata.payload, but processIncomingLeadMessage does not forward that metadata payload into processConversationMessage. Typed text and displayed titles may match some rules, but exact interactive payload navigation is not fully wired through this path. [backend/services/metaWebhookService.js:271] [backend/services/leadAiService.js:14]

10. API Security Observations

No active patient-to-LLM data transfer was found. This does not imply that no patient data leaves the application: a real WhatsApp transport would send text to Meta. Secret values were not copied into this report or archive. Local environment files, uploads, database backups and runtime test artifacts are excluded from the ZIP. .env.example is retained as a setup template. [.gitignore:2]

Webhook verification requires attention before public deployment: verifySignature exists, but returns true when the secret or signature header is missing. More importantly, the active handleWebhook controller delegates to handleMetaWebhookEvent without calling verifySignature. Express JSON parsing does not capture a signature-verification raw-body buffer here. These are source observations; no exploit or live request was attempted. [backend/services/metaWebhookService.js:31] [backend/controllers/leadController.js:32] [backend/server.js:40]

The webhook verification-failure log includes the submitted token. Development verification-token and JWT fallback constants exist in source; they are not printed here. Public simulator endpoints and permissive CORS should be reviewed before external exposure. No security configuration was changed in this documentation-only task. [backend/services/metaWebhookService.js:23] [backend/routes/leadRoutes.js:26] [backend/server.js:37]

11. Current Limitations

Natural-language support extends beyond menu buttons only through configured regex/keyword patterns and exact FAQ matching. It is not general language understanding or open-ended medical reasoning. Unrecognized questions receive the fixed unknown/staff message. Roman Urdu booking phrases are explicitly matched; much of the response language selection maps to English unless the stored language is urdu. [backend/services/conversationEngine.js:301] [backend/services/conversationEngine.js:397] [backend/services/conversationEngine.js:470] [backend/services/knowledgeService.js:45]

The optional LLM adapter is a disabled stub, not a working fallback. AI_PROVIDER and AI_MODEL do not select an implementation. Adding a real LLM would require deliberate code integration, credential management and clinical safeguards; this audit did not make those changes. [backend/services/conversationEngine.js:219]

Not Verified: production hosting configuration, deployed environment overrides, live Meta delivery, external provider account status, billing, model availability, production database contents and clinical adequacy of all stored knowledge. No provider endpoint was contacted and no outbound patient message was sent. Evidence applies to the inspected local project snapshot.

12. Final Technical Conclusion

The current Nighat Medical Complex patient assistant is rule-based and database-backed. No external AI API or inference model generates its replies. The exact configured identifier gemini-1.5-flash is present but unused; it must not be reported as an active Gemini integration. Website and WhatsApp intake share the same conversation engine, while WhatsApp delivery remains a separate transport concern.

> **Roman Urdu: Aap ka chatbot filhal kisi real AI API se jawab nahin banata. Yeh pehle se likhay huay messages, keywords aur MongoDB ki verified maloomat use karta hai. .env mein gemini-1.5-flash likha hai, lekin code us model ko call nahin karta. Booking state machine aur database ke zariye hoti hai.**

Deliverables: this PDF is saved in the project root as Nighat_Medical_Complex_AI_API_Audit.pdf. The updated source ZIP includes the PDF, application source, dependency lockfiles and setup template, but excludes live secrets, private uploads, backups and reinstallable dependency folders. Neither the PDF nor the ZIP is a database export.